

Project Reflection: Dual-Panel Activity & Protocol Visualizer

Generated on: 2026-09-17

1. AI Platform & Models Used

Platform Used: Google Antigravity

Model Used: Gemini 3.1 Pro (High)

Rationale for Choice:

Gemini 3.1 Pro (High) within the Google Antigravity environment was selected because of its agentic capabilities and advanced code generation context. It seamlessly creates a complete project structure, writing standard HTML, CSS, and Javascript simultaneously, whilst managing local files and dynamically fulfilling side requirements (like generating this exact PDF via Python execution). The agentic nature meant it could instantly prototype, test, and apply aesthetic UI patterns (Tailwind) in a single workflow without needing back-and-forth copy/pasting.

2. Synchronization Between Panels

The synchronization between the left Activity Panel and the right Protocol Visualization panel is achieved using an event-driven Finite State Machine in Vanilla JavaScript. When a user executes an action on the left (e.g., clicking 'Visit Page'), an event dispatcher triggers a specific protocol array (Flow Data) containing ordered message objects.

A global `currentStep` index tracks the state. A playback function using `setInterval` automatically increments this index every 2.5 seconds, injecting the specific message data (DNS, HTTP, SMTP) into the UI. CSS keyframe animations (transform and opacity) are tightly coupled with these state changes to render physical 'packets' moving across the screen synchronously with the protocol log updates.

3. Iterations, Corrections & What the AI Got Wrong

During development, the AI made structural decisions that required minor logical refinement:

- SMTP Initial Logic:** Initially, the AI generated a pure SMTP simulation but forgot that an email application must first resolve the MX (Mail Exchange) record before initiating the TCP connection to port 25. The instruction was updated to forcefully insert DNS resolution steps prior to the EHLO command.
- UI State Overlap:** Initially, if a user clicked 'Play' rapidly multiple times, the Javascript `setInterval` duplicated, causing packets to overlap and the timeline to sprint uncontrollably. The AI corrected this by tracking the interval ID and enforcing a strict `clearInterval()` call on any playback interruption or restart.

4. Key Differences in Protocol Flows

Browsing (DNS + HTTP):

A standard request/response model. The client asks for a single A record, gets an IP, and sends a monolithic GET request. The connection is stateless (though kept-alive at the transport layer).

Project Reflection: Dual-Panel Activity & Protocol Visualizer

Generated on: 2026-09-17

Mail (DNS + SMTP):

A highly conversational, stateful protocol. It requires querying an MX record rather than an A record. The subsequent SMTP communication requires constant acknowledgement (250 OK) at every phase: greeting, sender setup, recipient setup, and payload transmission, showing a much 'chattier' handshake than HTTP.

Streaming (DNS + HTTP HLS/DASH):

A tiered retrieval strategy. The client first fetches a 'Master Manifest' via HTTP, dissects it to find the requested quality tier, fetches a 'Media Playlist' for that tier, and finally loops through consecutive segmented video files (.ts). It demonstrates how modern streaming sits atop standard HTTP rather than using custom transport protocols.